BNN_FCC Contest Submission Report

April 17, 2026

Executive Summary

Submission status	Ready for evaluation. Standard contest verification passed, timing/area were collected with Openflex, and a supplemental coverage testbench is provided for evaluator coverage scripts.
Targeted tradeoff / use case	Maximize fmax for the contest SFC topology. The design targets a frequency-first streaming classifier for $784 \rightarrow 256 \rightarrow 256 \rightarrow 10$, accepting modest on-chip memory use to keep a short, timing-friendly pipeline while preserving AXI4-Stream protocol robustness.
Required contest verification	PASSED: all 50/50 tests completed successfully in `verification/bnn_fcc_tb.sv` under the contest stress settings `TOGGLE_DATA_OUT_READY=1`, `CONFIG_VALID_PROBABILITY=0.8`, and `DATA_IN_VALID_PROBABILITY=0.8`.
Coverage testbench for evaluator scripts	Use `verification/bnn_fcc_coverage_tb.sv` with `verification/cov_check.sh`. This supplemental testbench is separate from the standard contest testbench and is the one intended for Apple-style coverage evaluation.
Max frequency / clock period	452.694 MHz (2.209 ns).
Key resource use	LUT=9,426, REG=9,278, BRAM=16, URAM=14, DSP=0.
Measured stressed-run performance	Average latency=1076.6 cycles/image; average throughput=738.9 cycles/output; average throughput at the 100 MHz verification clock=135343.6 outputs/s.
Supplemental coverage result	54/54 coverage tests passed, 100.00% total covergroup coverage across 7/7 covergroup types, 89/89 instance bins hit, and zero assertion failures.

Important interpretation. The verification testbench intentionally injects input gaps and output backpressure, so the measured throughput is a stressed end-to-end number rather than a peak-throughput number. I therefore report both the raw measured values and simple estimates obtained by scaling the measured cycle counts by the achieved fmax.

Design Goal and Architecture Rationale

This submission is optimized for a system-level use case where timing closure and clock headroom are more important than minimizing every last LUT. The intended deployment model is a streaming image-classification accelerator that is configured once, then processes a long sequence of images while tolerating realistic AXI4-Stream stalls.

- **Top-level dataflow.** The top module `rtl/bnn_fcc.sv` accepts AXI4-Stream configuration data, binarizes 8-bit input pixels on the data input stream, stores intermediate layer activations, and performs a final argmax scan over the output-layer population counts before driving the output stream.
- **Layered compute pipeline.** Each neural-network layer is split into a `layer_memory` block

and a `compute_layer` block. `layer_memory` organizes weights and thresholds for burst-like access by parallel neuron lanes, while `compute_layer` sequences preload / compute / finish phases across those lanes.

- **Parallelism choice.** The timing wrapper `openflex/rtl/bnn_fcc_timing.sv` is configured with `PARALLEL_INPUTS=8` and `PARALLEL_NEURONS={8,8,10}`. For the contest topology, this means the two 256-neuron hidden layers are processed in batches of 8 neurons, while the 10-neuron output layer is processed in a single 10-neuron batch.
- **Why this fits the objective.** The design keeps configuration parsing, memory access, neuron accumulation, and argmax handling in clearly staged blocks. That organization uses more dedicated memory than a fully serialized implementation, but it shortens the critical work per cycle and aligns with the stated goal of maximizing clock frequency.

Timing, Area, and Measured Performance

Timing and utilization were collected with Openflex/Vivado using out-of-context synthesis and implementation of `bnn_fcc_timing`, targeting XCU250-FIGD2104-2L-E.

Metric	Used	Total	Utilization
fmax	452.694 MHz (2.209 ns period)		
LUT	9,426	1,728,000	0.545%
REG	9,278	3,456,000	0.268%
BRAM	16	2,688	0.595%
URAM	14	1,280	1.094%
DSP	0	12,288	0.000%

The standard contest verification flow used `verification/bnn_fcc_tb.sv` with a 100 MHz testbench clock and the stress settings above. The run completed successfully and reported the following measured performance:

Metric	Value
Average latency (cycles per image)	1076.6
Average latency at 100 MHz	10766.0 ns
Average throughput (cycles per output)	738.9
Average throughput at 100 MHz	135343.6 outputs/s
Estimated throughput at achieved fmax	612658.7 outputs/s
Estimated latency at achieved fmax	2.378 μs/image

Standard Contest Verification

The required contest verification result comes from the checked-in Openflex/Questa run driven by `openflex/verify.sh`.

- **Testbench and workload.** `verification/bnn_fcc_tb.sv` with the trained MNIST contest topology $784 \rightarrow 256 \rightarrow 256 \rightarrow 10$.

- **Tool provenance.** QuestaSim-64 2023.3 (qrun/vlog/vopt/vsim).
- **Outcome.** PASSED, with all 50 tests completed successfully and no simulation errors reported in the run log.
- **Interpretation.** This is the "required tests" result for submission grading. The coverage testbench discussed below is supplemental and should be treated as additional verification evidence rather than a replacement for the standard testbench.

Module-level verification To address the README judging criterion around unit-level verification, the repository includes self-checking benches for `config_parser`, `layer_memory`, `neural_processor`, and `compute_layer`. Additional edge/stress benches target parser operation at `CONFIG\_BUS\_WIDTH=32`, small or degenerate memory sizing (for example `WEIGHT\_MEM\_DEPTH=1`), `N=1` popcount behavior, and multi-chunk layer operation with backpressure. The full unit-test suite is executed by `verification/run_unit_tests.sh`.

Coverage Methodology and Coverage-Plan Mapping

For Apple-style coverage evaluation, use `verification/bnn_fcc_coverage_tb.sv` and run `verification/cov_check.sh`. The bench switches to the smaller custom topology $13 \rightarrow 10 \rightarrow 10 \rightarrow 5$, which intentionally forces partial final beats on the 64-bit input stream so that `TKEEP` handling is exercised in a controlled way. The run completed 54/54 checks successfully and reached 100.00% total covergroup coverage across all 7 covergroup types.

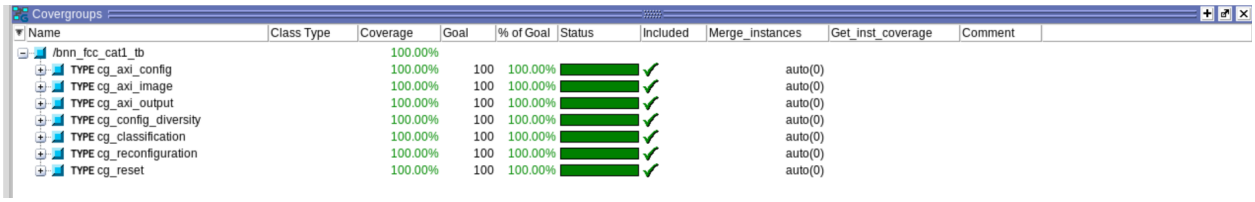

Figure 1: Supplemental coverage run summary. All seven coverage-plan covergroup types reached 100% coverage.

Coverage plan item	**How it is addressed in `verification/bnn_fcc_coverage_tb.sv`**	**Evidence**
Category 1: AXI4-Stream protocol patterns	`update_probabilities()` rotates configuration and image valid patterns between continuous, intermittent, and bursty modes. The output-ready driver rotates between always-ready, intermittent ready, and long backpressure bursts. The 13-input custom topology forces partial input beats, and the sequencer explicitly exercises `TLAST` boundaries.	Config AXI 9/9; image AXI 10/10; output AXI 8/8.
Category 2: configuration data diversity	`create_diverse_config()` constructs all-zero, sparse, balanced, dense, and all-one weight patterns, plus small / medium / large threshold classes. Threshold "don't care" fields are intentionally randomized in part of the configuration stream.	Configuration diversity 31/31.
Category 3: computational stimulus	`retune_model_for_cat3()` synthesizes examples that hit every output class in the custom 5-class topology. A directed repeated/varying class sequence is prepended, then 50 random vectors are added to broaden data-space exploration.	Classification 7/7.
Category 4: configuration-image sequencing	The sequencer performs an initial partial configuration, a full reconfiguration, weights-only partial reconfiguration, thresholds-only partial reconfiguration, weights+thresholds partial reconfiguration, and an additional full reconfiguration while image traffic is being exercised.	Reconfiguration 12/12.
Category 5: reset scenarios	Resets are injected before normal traffic, during configuration, on an image `TLAST` boundary, during output draining, and after the output stream has drained. Both same-configuration and different-configuration post-reset cases are covered, along with cold/warm/hot workload buckets.	Reset 12/12.

Across the seven covergroup instances, all 89/89 bins were hit: 9/9 for configuration AXI behavior, 10/10 for image AXI behavior, 8/8 for output AXI behavior, 31/31 for configuration diversity, 7/7 for class coverage, 12/12 for reconfiguration behavior, and 12/12 for reset behavior. The coverage report also records zero assertion failures.

Code Coverage

The filtered code-coverage summary for the supplemental coverage run is 71.91%. For the main RTL files, the branch-coverage breakdown is:

Module	Branch coverage
`rtl/bnn_fcc.sv`	85.36%
`rtl/compute_layer.sv`	92.30%
`rtl/config_parser.sv`	81.81%
`rtl/layer_memory.sv`	85.18%
`rtl/neural_processor.sv`	90.00%

The remaining misses are concentrated in generic or defensive branches that are not central to the contest operating point: elaboration-time parameter guards, fallback paths for unused parameterizations, and generic bounds/default handling in the parameterized parser / memory / argmax logic. In other words, the directed coverage-plan scenarios are fully exercised, while some reusable safety logic remains naturally unhit under the contest topologies and parameter settings.

Design Space Exploration Summary

The file `syn/results.csv` records the frequency sweep across multiple implementation checkpoints. The CSV uses informal internal pass labels, so the official report summarizes the top results with descriptive names instead. The top entries by fmax are:

Sweep point	fmax	LUT	REG	BRAM	URAM
Final submission configuration	435.730	9624	8789	16	14
Alternate high-fmax configuration	432.339	9564	8795	16	14
Pipelined XNOR exploration	426.076	10723	10356	16	14
Intermediate timing checkpoint	423.191	9237	8334	16	14
Earlier baseline configuration	417.711	9149	8146	16	14

The final submission configuration ties a second late-stage candidate for the highest reported fmax in the sweep. Relative to the earlier baseline configuration, the final submission configuration improves frequency by 18.019 MHz, which is a 4.31% gain, while keeping the same BRAM and URAM counts as the co-best result.

Reproducibility

Standard contest verification

```
cd openflex
./verify.sh
```

Timing and area collection

```
cd openflex
openflex bnn_fcc_timing.yml -c bnn_fcc.csv
```

Supplemental coverage run

```
cd verification
./cov_check.sh
```

Module-level unit tests

```
cd verification
./run_unit_tests.sh
```

Evaluator note. If your coverage scripts require an explicit top-level testbench, point them at `verification/bnn_fcc_coverage_tb.sv` and use `verification/cov_check.sh`. The standard contest result should still be evaluated with `verification/bnn_fcc_tb.sv`.

Tool Versions

- Vivado v2021.2
- QuestaSim-64 2023.3 (qrun/vlog/vopt/vsim)